PROJECT WRITE-UP

Retail Data Pipeline

Ingestion, validation and reconciliation

What the pipeline does, the problem behind it,
and the decisions I made while building it.

Python | SQL | SQLite / PostgreSQL | pandas | SQLAlchemy | pytest

Summary

A pipeline that collects order, customer, inventory and payment data from separate systems, keeps bad data out, checks whether every order was actually paid correctly, and produces a clean database, a list of problems for someone to fix, and some analytics on top.

3,060	45	62	65
records processed per run	bad records caught and quarantined	reconciliation exceptions found	automated tests all passing

Contents: 1. Summary 2. The real-world problem 3. How the pipeline works 4. Data quality 5. Reconciliation 6. Results 7. Analytics 8. Engineering qualities 9. Industry use 10. Limitations and next steps Glossary

1. Summary

Every business that sells online runs on several systems at once. Orders are captured by a store platform, customer details live in a CRM, money moves through a payment provider, and stock is tracked by a warehouse system. Each system is built and owned by a different team or vendor, and they never agree perfectly. Some records are malformed, some are duplicated, and some refer to things that don't exist in the other systems. Most importantly, the money recorded by the payment provider does not always match what the order system says was sold.

I built this project to work through that problem end to end. It brings the four sources together and pushes every record through a series of checkpoints:

- **Ingestion and staging** keep an exact, traceable copy of what each source sent.
- **Validation** tests every record against business rules. Bad records are not deleted and do not crash the run. They are quarantined in an error table along with the reason they failed.
- **Transformation** cleans and standardises the good records into a consistent model.
- **Reconciliation** cross-checks orders against payments and the product catalog to find missing payments, double charges, wrong amounts, charges on cancelled orders and payments with no order behind them, while allowing a settlement window for payments that are simply still on their way.
- **Loading and reporting** deliver a clean database for analysis and a prioritised exception report that tells people exactly what to fix.

To be able to test it properly, I generate the sample data with known problems planted in it. On the sample of about 3,000 records, the pipeline catches all 45 deliberately planted data errors and all 62 planted reconciliation problems, flagging around $16,450 in payment discrepancies against roughly $697,000 of revenue. It also does this repeatably. Running it again gives exactly the same result, and every run is logged. Payments can come from a bundled offline mock API or from a real Paystack test account, and both find exactly the same problems.

> **Why this matters:** without a process like this, these problems are usually found late (by an angry customer, an auditor, or a month-end spreadsheet exercise) or not at all. The pipeline turns them into a short, prioritised list on the day the data arrives.

2. The real-world problem

2.1 Data lives in many systems that don't agree

The four inputs in this project represent systems found in almost every retail or e-commerce business:

Source in project	Real-world equivalent	Typical owner	How data usually arrives
Orders CSV	E-commerce platform or order management system	Sales / operations	Nightly export or database extract
Customers CSV	CRM or customer database	Marketing / customer service	Export or API
Payment API	Payment gateway, e.g. Paystack or Stripe	Finance (external vendor)	Provider's REST API, one page at a time
Inventory CSV	Warehouse management or ERP system	Operations / supply chain	Scheduled file drop

Because these systems are built independently, they disagree in predictable ways. People type data in by hand, so emails get mistyped and fields get left blank. Integrations retry after timeouts, so the same record gets sent twice. Dates arrive in impossible formats. A customer can be charged twice because a checkout page was submitted twice. An order can be cancelled after the card was already charged. A price change can reach the catalog but not the order system.

2.2 What goes wrong if nobody checks

Problem	Who feels it	Business consequence
Customer charged twice	Customer, support team	Refund costs, complaints, possible chargebacks and dispute fees, damage to trust and reviews
Order shipped but never paid	Finance	Revenue leakage: goods leave the warehouse with no money coming in
Cancelled order still charged	Customer, finance	Refund obligation, customer complaints, potential consumer-protection issues
Payment received with no matching order	Finance, audit	Money that can't be explained; could signal an integration fault, test transactions or fraud
Paid amount differs from order amount	Finance	Under- or over-charging; books that don't balance
Invalid or duplicate customer records	Marketing, support	Receipts and notifications that never arrive, inflated customer counts
Reports built on unchecked data	Management	Decisions based on wrong revenue, wrong customer counts and wrong product performance

2.3 How it is often handled today

In many organisations, especially smaller ones, reconciliation is a manual month-end task. An analyst exports files from each system into a spreadsheet, uses lookups to match orders to payments, and scrolls for anything that looks wrong. This approach has well-known weaknesses:

- **It's slow.** It takes hours or days, so problems are found weeks after they happened.
- **It's error-prone.** One broken formula or sorted column can hide a problem.
- **It doesn't scale.** What works for 500 orders fails at 50,000.
- **It leaves no audit trail.** It is hard to prove later what was checked and what was found.
- **It depends on one person.** When the person who knows the spreadsheet is away, the check doesn't happen.

The idea behind this project is to do those same checks in code instead: the same way every time, in seconds, with a record of everything that was found.

3. How the pipeline works

Data flows through a fixed sequence of stages. I kept each stage to one job in its own Python module, handing its output to the next, which made each part much easier to test on its own.

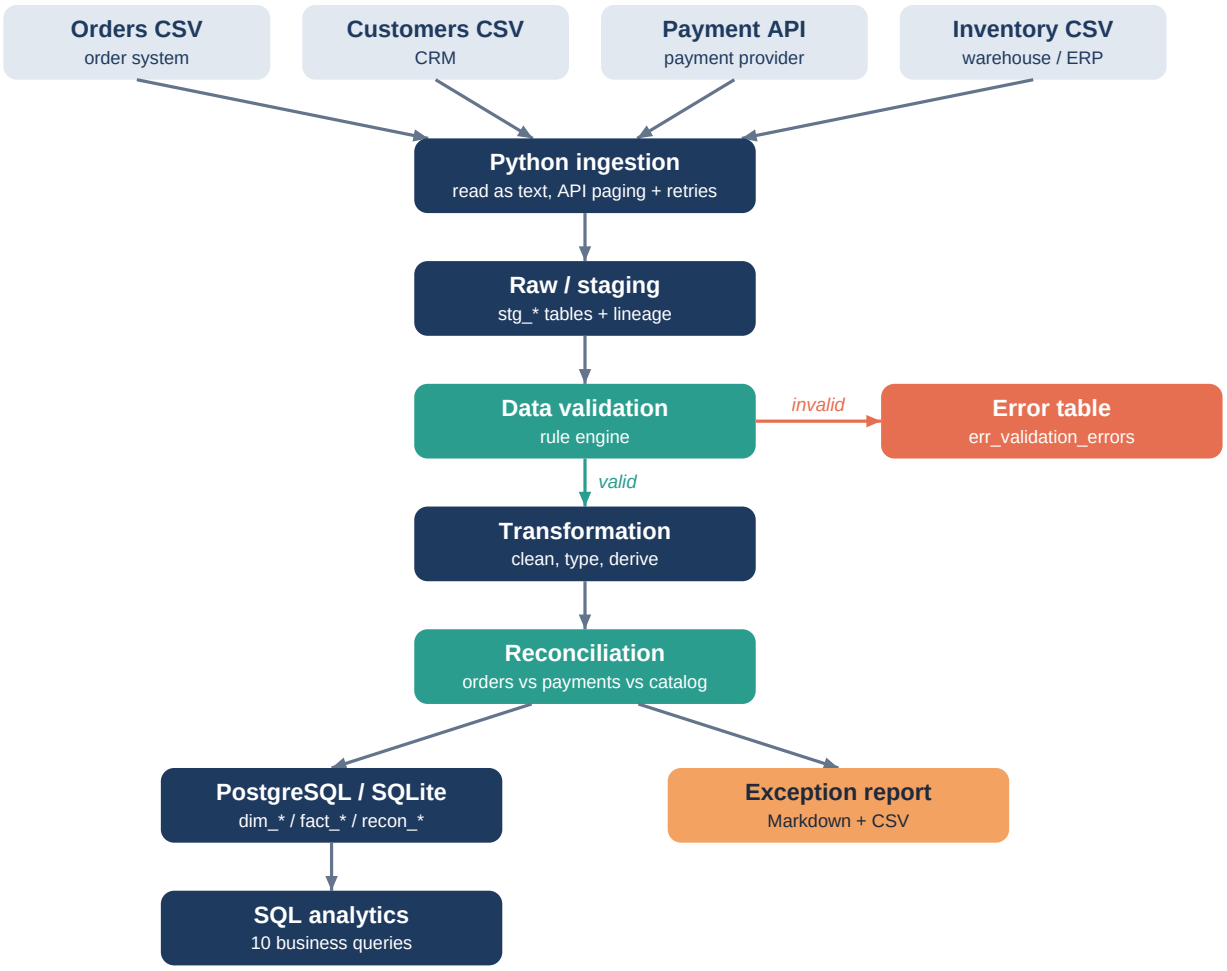


Figure 1. Pipeline architecture. Teal stages make decisions about data; coral and amber outputs are where problems are surfaced to people.

Stage	What it does	Real-world value
Ingestion	Reads the three CSV files and pages through the payment API (an offline mock or a real Paystack test account), retrying automatically on temporary failures and rate limits. Everything is read as plain text.	Reliable collection even when a vendor's API is flaky. Nothing is silently altered on the way in.
Raw / staging	Stores each source exactly as received, stamped with a batch ID, source name and timestamp.	Lineage: any number in a report can be traced back to the raw record and the run that produced it.
Validation	Applies business rules to every record. Failures go to the error table with the rule broken and the full original record.	Bad data is stopped at the door without stopping the business. Problems are visible, explainable and fixable at the source.
Transformation	Trims and standardises text, converts types, and calculates order totals and reporting months.	Consistent, analysis-ready data. 'ZA', 'za' and ' ZA ' all become one country.

Stage	What it does	Real-world value
Reconciliation	Matches orders to payments and to catalog prices, allows a settlement window for payments in transit, labels every order and payment, and computes financial control totals.	Proof that the money adds up, plus a precise list of what doesn't.
Load	Writes the clean model to the database in a single transaction.	The database is never left half-updated, and re-running is always safe.
Exception report	Produces a readable report and CSV work lists.	A ready-made to-do list for finance and operations, ordered by severity.
SQL analytics	Answers business questions directly from the trusted tables.	Decisions are based on data that has already been checked.

4. Data quality: validation and the error table

A simple pipeline usually handles bad data in one of two ways. It crashes, which stops everything over one typo. Or it silently drops or 'fixes' the record, so the problem disappears without anyone knowing. I went with the third option, which is the usual approach in data engineering: **quarantine**. Each bad record is set aside with an explanation, and everything else carries on.

4.1 The rules

Source	Checks applied	Example of what it catches
Customers	Required fields, unique ID, email format, two-letter country code, valid date	An email of *john.smith@*; a sign-up date of 30 February
Inventory	Required fields, unique ID, price above zero, stock is a whole number, valid date	A price of -19.99; a stock level of 'lots'
Orders	Required fields, unique ID, customer and product must exist **and** be valid, quantity is a whole number above zero, known status	An order for customer C9999 who doesn't exist; a quantity of -2; a status of 'lost'
Payments	Required fields, unique ID, amount above zero, supported currency, known method and status, valid timestamp	A missing amount; currency 'XYZ'; a timestamp of 'yesterday'

4.2 What a quarantined record looks like

Every failure becomes one row in `err_validation_errors`. Here are real rows from the sample run:

Source	Record	Rule broken	Column	Bad value	Message
customers	C0091	not_null	email	(blank)	email is required
customers	C0187	duplicate_key	customer_id	C0187	appears more than once; first occurrence kept
orders	(various)	unknown_reference	customer_id	C9999	does not match any valid customer
payments	(various)	invalid_value	currency	XYZ	must be one of ['usd']

Each row also stores the complete original record as JSON. A data steward can see exactly what arrived, which rule it broke and which run it came from, then fix it in the source system so it flows through cleanly next time.

4.3 Why the order of checks matters

Customers and products are validated *before* orders, and an order is only accepted if its customer and product *passed* validation. This mirrors how a real database protects itself with foreign keys: you can't record a sale to a customer who doesn't exist. Checking in dependency order means the final load can never break the database's own integrity rules.

> **Real-world parallel:** this is the same idea as tools such as Great Expectations, Soda or dbt tests, which data teams use to write rules about what 'good' data looks like and to stop bad data reaching dashboards.

5. Reconciliation: proving the money adds up

Validation asks whether each record is well-formed *on its own*. Reconciliation asks whether records from different systems *agree with each other*. A payment can be perfectly valid (correct format, positive amount, real currency) and still be wrong because it was taken twice, or because there is no order behind it.

The idea is the same as reconciling a bank statement against your own records: every transaction on one side should have exactly one matching transaction on the other, for the same amount. Anything that doesn't line up is an **exception** that needs a human decision.

5.1 The exceptions the pipeline detects

Exception	Severity	Likely real-world cause	Recommended action
Missing payment	High	Payment failed silently; order was released before payment cleared. Only raised once the order is older than the settlement window	Hold or chase the order; investigate the payment integration
Duplicate payment	High	Customer double-clicked checkout; a retry after a timeout charged again	Refund the extra charge before the customer disputes it
Amount mismatch	High	Discount or shipping applied in one system only; currency rounding	Correct the order or issue a partial refund or charge
Orphan payment	High	Test transactions, deleted orders, ID changes, or possible fraud	Investigate urgently; money that can't be explained is an audit risk
Charged on cancelled order	Medium	Cancellation didn't trigger a refund	Refund the customer
Payment for rejected order	Medium	The order exists but has a data error (e.g. unknown customer)	Fix the order data at the source; the payment will then match
Price mismatch	Low	Promotion, manual override or stale catalog price	Confirm the discount was authorised

Two situations are deliberately **not** flagged, because a good reconciliation process must avoid false alarms as carefully as it finds real problems, or people learn to ignore it:

- **Failed attempts followed by a successful retry.** That is normal customer behaviour.
- **Payments still in transit.** Card and EFT payments often land a day or two after the order. A completed order with no payment is tagged *awaiting settlement* until it is older than the settlement window (2 days by default, configurable), and only then raised as a missing payment. The sample data includes five such recent orders, worth $2,377.42, which are reported separately instead of as errors.

5.2 A worked example

From the sample run: order **O00581** was for $1,756.16. The payment provider shows two successful payments, PAY000484 and PAY000485, two minutes apart, totalling $3,512.32. The pipeline marks the order and both payments as DUPLICATE_PAYMENT with a difference of +$1,756.16 and severity *high*. A finance team member can refund one charge the same day, before the customer notices and raises a dispute with their bank.

5.3 Control totals

Beyond individual exceptions, the pipeline produces summary **control totals**, a standard accounting check that the overall numbers tie out:

Control total	Sample value	Meaning
Expected revenue (completed + shipped orders)	$698,965.08	Everything the order system says was sold
Within settlement window	$2,377.42	Recent orders whose payments may still be on their way
Expected revenue due	$696,587.66	What should have been collected by now
Settled payments on valid orders	$696,533.32	What the payment provider actually collected for those orders
Settled payments with no valid order	$4,348.53	Money received that can't be tied to a valid order
Net difference	-$54.34	The small headline gap hides much larger offsetting errors

> **Key insight:** the headline totals differ by only $54. A manager glancing at those totals would conclude everything is fine. In reality, about $6,000 of orders were never paid and about $4,500 was double-charged; the two errors almost cancel out. This is exactly why record-level reconciliation is needed, and why totals alone are not enough.

6. Results from the sample run

The sample data is synthetic. It was generated with a fixed random seed and contains a known set of planted problems, recorded in a *defect manifest*. This makes the results checkable: the tests confirm that what the pipeline finds matches what was planted, exactly. The rates below reflect the planted defects, not industry benchmarks.

Source	Ingested	Valid	Rejected	Loaded	Rejection rate
Customers	202	192	10	192	5.0%
Inventory	41	37	4	37	9.8%
Orders	1,507	1,485	22	1,485	1.5%
Payments	1,310	1,301	9	1,301	0.7%
Total	**3,060**	**3,015**	**45**	**3,015**	**1.5%**

Every record is accounted for: ingested = valid + rejected, and valid = loaded, for every source. Nothing disappears without a trace.

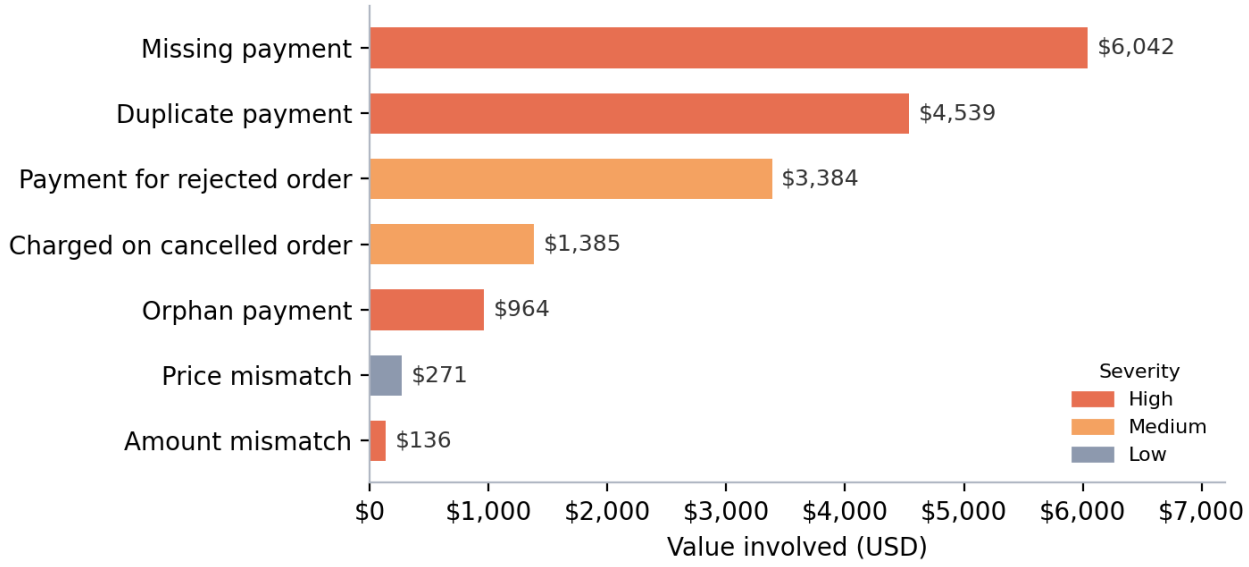


Figure 2. Value involved in each type of reconciliation exception (62 exceptions in total).

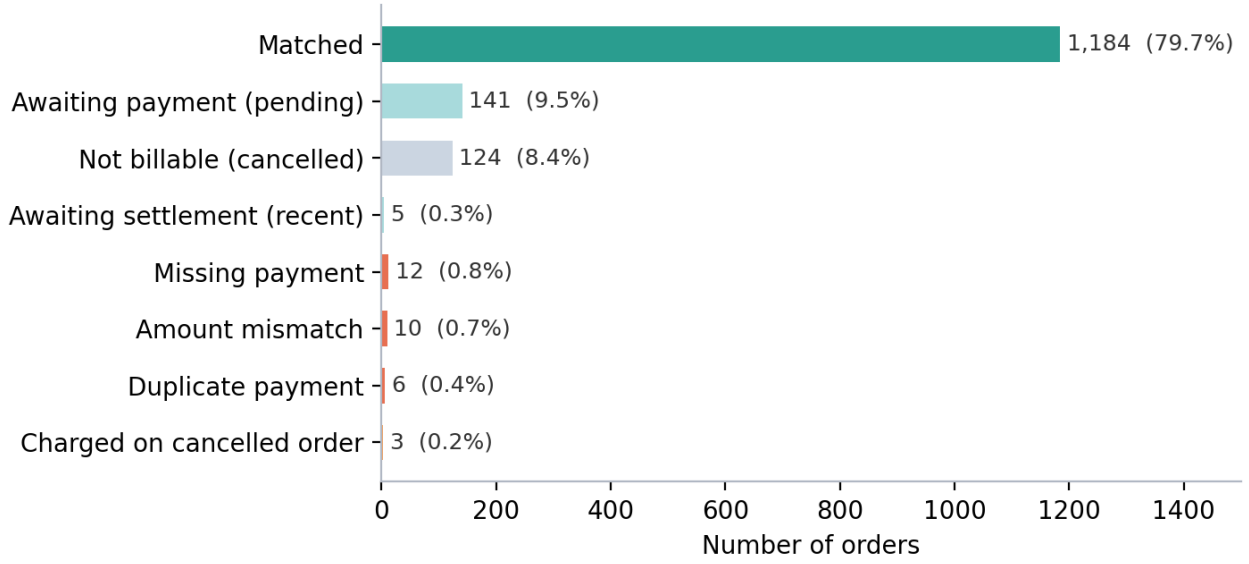


Figure 3. Reconciliation status of all 1,485 loaded orders. About 80% reconcile cleanly; pending, cancelled and very recent orders are correctly treated as not yet due rather than flagged.

7. Analytics: from clean data to decisions

Once data is validated and reconciled, it can be trusted for reporting. The project includes ten SQL queries that answer the questions different teams typically ask. The same SQL runs unchanged on SQLite and PostgreSQL.

Query	Business question	Typical audience	SQL techniques
Monthly revenue	How is revenue trending, and how fast is it growing month to month?	Leadership, finance	CTE, running total, LAG()
Top customers	Who are our most valuable customers?	Sales, marketing	CTE, RANK()
Category performance	Which categories drive revenue, and what share does each have?	Merchandising	Window share of total

Query	Business question	Typical audience	SQL techniques
Best seller per category	What is the top product in each category?	Buying, inventory	ROW_NUMBER() per partition
Payment method mix	How do customers pay, and which methods fail most?	Finance, product	Conditional aggregation
Revenue by country	Where are our customers and how often do they buy?	Marketing, expansion	Joins, ratios
Order reconciliation status	What share of orders reconcile cleanly, and how much money is at risk?	Finance	Window percentages
Reconciliation exceptions	What needs fixing, by severity and value?	Finance ops	Filtering latest batch
Data quality by rule	Which sources and rules produce the most errors?	Data owners	Grouping, DISTINCT
Pipeline run history	Is the pipeline healthy over time?	Data engineering	Audit log reporting

The last three queries are worth highlighting. They don't analyse sales; they analyse the *data and the pipeline itself*. Monitoring data quality over time is how a team notices that a source system has started sending worse data, often before anyone downstream is affected.

8. Engineering qualities that make it trustworthy

A pipeline is only useful if people can rely on it. These are the choices I made to move it beyond a one-off script:

Quality	How the project achieves it	Why a business cares
Idempotent	Each run fully refreshes the core tables, so running twice gives the same result. A test proves it.	If a run is repeated by accident or after a failure, no data is duplicated.
Transactional	The core load happens in one database transaction.	A failure halfway leaves the previous good data in place, never a half-updated mix.
Auditable	Every run is logged with status and counts; errors and exceptions keep their history by batch.	You can prove to an auditor what was checked, when, and what was found.
Traceable (lineage)	Raw data is kept as received, with batch, source and timestamp.	Any figure can be traced back to the exact source record.
Resilient	API calls retry with exponential backoff; permanent errors fail fast with a clear message.	Temporary vendor outages don't need a person to restart the job.
Tested	65 automated tests, including end-to-end runs checked against the defect manifest in both mock and Paystack mode; CI runs them on every change.	Changes can be made with confidence that nothing else broke.
Portable	Standard SQL that runs on SQLite and PostgreSQL; configuration through environment variables.	Runs on a laptop for development and on a server database in production.

9. Where this pattern is used in industry

The retail example is one instance of a pattern found wherever two systems must agree about money, stock or obligations:

Industry	What gets reconciled
E-commerce and retail	Orders vs payment provider settlements vs warehouse shipments
Banking and fintech	Internal ledgers vs card networks, bank statements and settlement files
Insurance	Claims approved vs payouts actually made
Healthcare	Claims submitted vs payments and remittances received from insurers
Logistics	Shipments delivered vs carrier invoices
Subscription software	Active subscriptions vs billing system charges

9.1 How the project maps to production tools

I built this with plain Python and SQL so every step is visible and it runs on any laptop. In larger organisations each stage is often handled by a specialised tool, but the concepts are the same:

Stage in this project	Built here with	Commonly used tools at scale
Orchestration	orchestrator.py	Apache Airflow, Dagster, Prefect
Ingestion	pandas, requests, Paystack API	Fivetran, Airbyte, custom Python services
Validation	Custom rule engine	Great Expectations, Soda, dbt tests
Transformation	pandas	dbt, Apache Spark
Storage	SQLite / PostgreSQL	PostgreSQL, Snowflake, BigQuery, Amazon Redshift
Reporting	Markdown report, SQL	Power BI, Tableau, Looker
Testing and CI	pytest, GitHub Actions	pytest, GitHub Actions, GitLab CI

Part of my reason for building it this way was to understand what those tools are doing underneath: staging, validation, idempotent loads, reconciliation and lineage.

10. Limitations and next steps

There's plenty this doesn't do yet. If I were taking it further, or running it for a real business, these are the things I'd work on next:

- **Full refresh instead of incremental loads.** The sources are treated as complete snapshots. At larger volumes, the pipeline would load only new or changed records and use upserts.
- **Mostly synthetic data.** The Paystack connector shows the pipeline working against a real payment API, but the orders and their error rates are still generated. Real sources would bring messier, less predictable problems.
- **Transactions only, not payouts.** Paystack also reports settlements (money paid out to the bank) and fees. A fuller version would reconcile those too, so the bank balance ties back to individual orders.
- **Single currency.** Real payment data often spans currencies and needs exchange rates and rounding rules.

- **No scheduler.** The pipeline runs on demand. In production it would run on a schedule, for example nightly through cron, Windows Task Scheduler or Airflow.
- **No quality thresholds or alerts.** A production run might fail, or send an alert, if the rejection rate suddenly rose above a set level, for example 5%.

Glossary

Term	Meaning
Pipeline	An automated sequence of steps that moves data from sources to a destination, transforming it on the way.
Staging	A landing area that holds data exactly as received, before it is checked or changed.
Validation	Checking that each record follows defined rules (required fields, formats, ranges, references).
Quarantine / error table	Where invalid records are stored with the reason they failed, instead of being deleted.
Referential integrity	The rule that a reference must point at something that exists, e.g. an order's customer.
Reconciliation	Comparing records from two or more systems to confirm they agree, and listing differences.
Control totals	Summary figures (counts, sums) compared between systems to confirm nothing was lost or added.
Idempotent	Safe to run more than once: repeating the run gives the same outcome.
Transaction	A group of database changes that either all succeed or all fail together.
Lineage	The ability to trace data back to where it came from and which run produced it.
Settlement window	The grace period allowed for a payment to arrive before an unpaid order is treated as a problem.
Star schema	A reporting model with fact tables (events such as orders) linked to dimension tables (customers, products).